Лабораторна робота №8

Перенесення навчання (Transfer Learning)

Виконав: студент групи 443-4 **Бабин Богдан Олегович**

Мета роботи

Навчитися використовувати попередньо навчені на ImageNet мережі (VGG16, InceptionV3, ResNet50) для власної задачі «кіт / собака»: взяти їхню згорткову частину як готовий виділювач ознак, заморозити її і навчити поверх неї невеликий класифікатор. Дослідити вплив Dropout на перенавчання і порівняти результат з власною CNN з лабораторної 7.

Посилання

- 8.1 VGG16: https://colab.research.google.com/drive/1_Mjr1TrSe-G9owdef-ZDj-LdWgSQGdCE?usp=sharing
- 8.2 InceptionV3: https://colab.research.google.com/drive/1JcG65q69NEc_wKTqSHek5rCjSwNx2w6i?usp=sharing
- 8.3 ResNet50: https://colab.research.google.com/drive/1YaKoZcwKop29RA1PoeCCUmyH8iL9yCd1?usp=sharing
- Папка на Google Drive (ноутбуки з результатами): https://drive.google.com/drive/folders/1it0cYh2vOXijSTtckwTBZqJOLYoU0HgK?usp=sharing

Код оновлено до Keras 3: `Input(shape=...)` першим шаром Sequential, `tensorflow.keras.utils.load_img`, збереження у форматі `.keras`. Навчання — у Google Colab на GPU T4.

1. Що таке перенесення навчання

Мережа, навчена на ImageNet (1,2 млн фото, 1000 класів), уже вміє виділяти загальні ознаки зображень: перші шари — краї й кольори, середні — текстури й форми, останні — частини об’єктів (очі, вуха, шерсть). Ці ознаки підходять і для котів та собак, тому:

- беремо лише **згорткову частину** (`include_top=False`) — без класифікатора на 1000 класів;
- **заморожуємо** її (`base_net.trainable = False`) — ваги ImageNet не змінюються;
- додаємо свій класифікатор: Flatten/GlobalAveragePooling → Dense 256 (ReLU) → **Dropout** → Dense 1 (sigmoid);
- зображення обробляємо функцією `preprocess_input` саме цієї мережі (так само, як при навчанні на ImageNet), а не `rescale=1/255`.

Навчаються лише кілька сотень тисяч – 2 млн параметрів класифікатора, тому навчання швидке (~2 хв на модель) і не потребує великого набору даних.

```
base_net = VGG16(weights='imagenet', include_top=False, input_shape=(150, 150, 3))
base_net.trainable = False                     # заморожуємо згорткову частину

def build_model(dropout):
    model = Sequential([Input(shape=(150, 150, 3)), base_net,
                        Flatten(),              # для InceptionV3 / ResNet50 — GlobalAveragePooling2D()
```

```
                    Dense(256, activation='relu'), Dropout(dropout),
                    Dense(1, activation='sigmoid')])
    model.compile(loss='binary_crossentropy', optimizer=Adam(learning_rate=1e-4), metrics=['accuracy'])
    return model
```

2. Підготовка даних

Набір Dogs vs Cats (дзеркало Microsoft, той самий набір що й на Kaggle) підготовлено скриптом `prepare_dataset.py` з лабораторної 7: відкинуто пошкоджені файли, дані перемішано і поділено 70 / 15 / 15. Для перенесення навчання взято **3 000 зображень кожного класу** — згорткова частина вже навчена, тож цього достатньо, а 12 експериментів (3 мережі × 4 Dropout) встигають за ~30 хв.

```
cats: знайдено 12500, пошкоджених відкинуто 1
dogs: знайдено 12500, пошкоджених відкинуто 1
Обмеження: 3000 зображень кожного класу
dataset/train: cats = 2100, dogs = 2100
dataset/val:   cats = 450,  dogs = 450
dataset/test:  cats = 450,  dogs = 450
```

3. Моделі

Мережа	Шарів у згортковій частині	Вихід згорткової частини	Перехід до Dense	Усього параметрів	Навчаються
VGG16	19	4×4×512	Flatten → 8 192	16 812 353	2 097 665
InceptionV3	311	3×3×2048	GAP → 2 048	22 327 585	524 801
ResNet50	175	5×5×2048	GAP → 2 048	24 112 513	524 801

Для InceptionV3 і ResNet50 використано **GlobalAveragePooling2D** (усереднення кожної карти ознак до одного числа): карта ознак має 2048 каналів, і Flatten дав би десятки мільйонів параметрів у Dense. Для VGG16 — Flatten, як у лекції. Оптимізатор Adam з малою швидкістю навчання 1e-4, 8 епох, batch 32, зображення 150×150.

4. VGG16: вплив Dropout на перенавчання

Навчено 4 моделі з Dropout = 0; 0,3; 0,5; 0,7 (однаковий seed, 8 епох без ранньої зупинки — щоб було видно перенавчання).

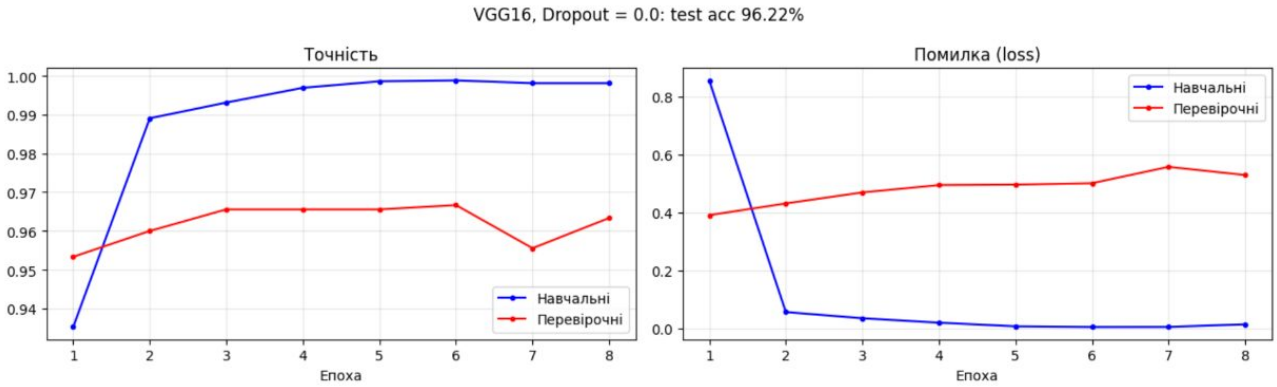


Рис. 1. VGG16, Dropout = 0: навчальна точність ~100%, втрати на перевірці ростуть з 0,39 до 0,53

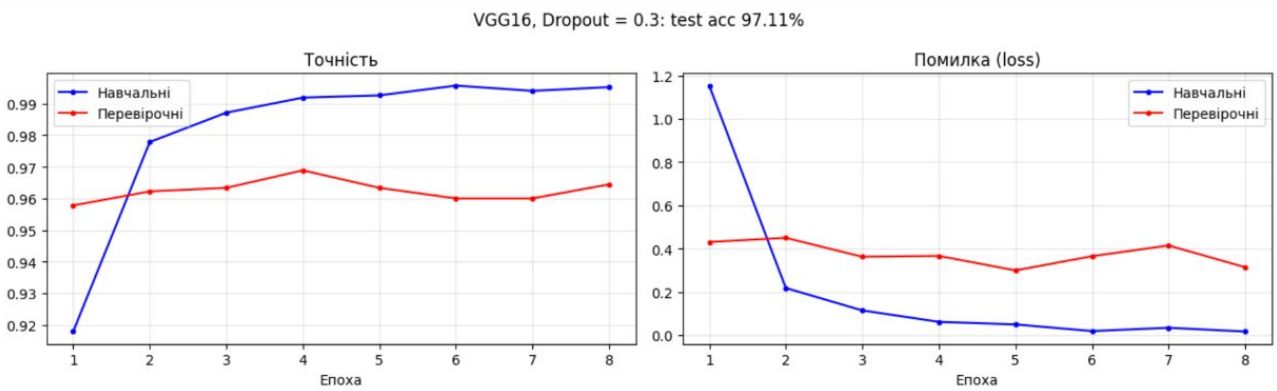


Рис. 2. VGG16, Dropout = 0,3

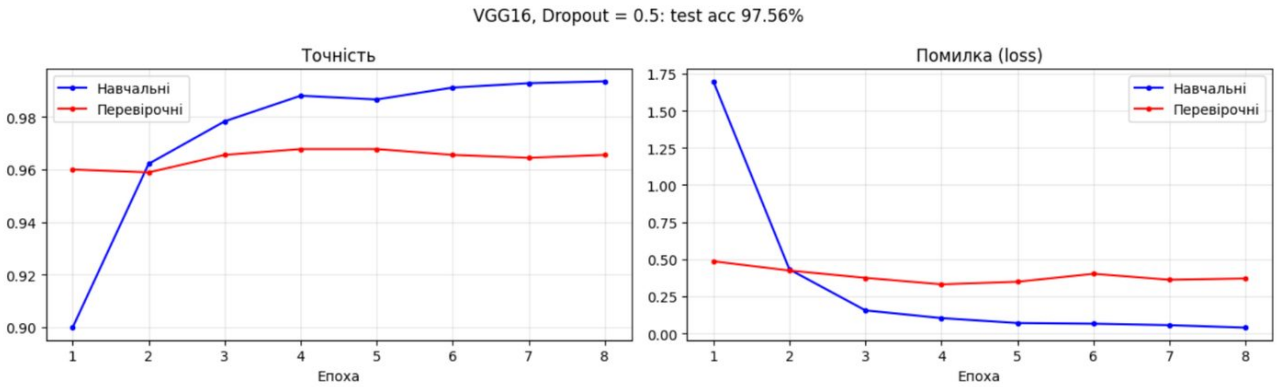


Рис. 3. VGG16, Dropout = 0,5 — найкраща точність на тесті 97,56%

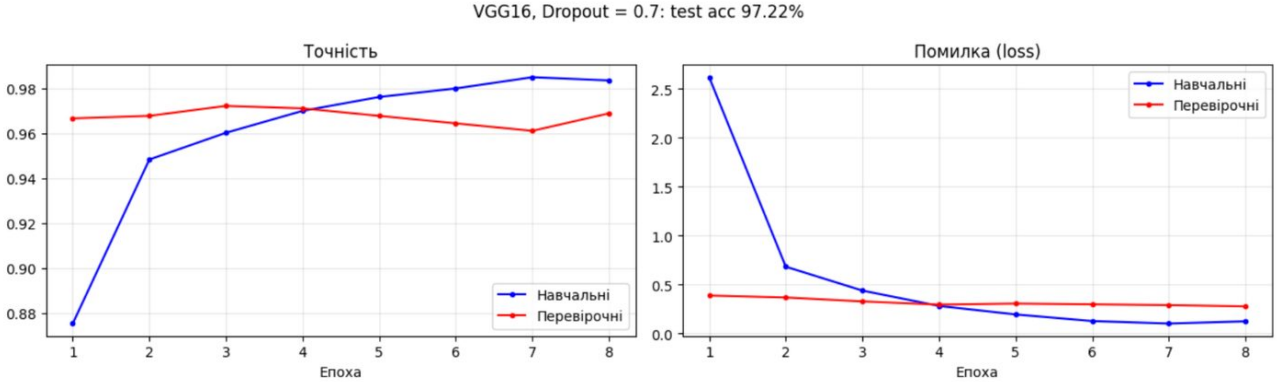


Рис. 4. VGG16, Dropout = 0,7: криві навчання і перевірки майже зійшлися

Dropout	Train acc, %	Val acc, %	Розрив train−val	Train loss	Val loss	Test acc, %	Test loss	Час, с
0,0	99,81	96,33	3,48	0,0135	0,5290	96,22	0,5500	136
0,3	99,52	96,44	3,08	0,0166	0,3138	97,11	0,4694	128
0,5	99,36	96,56	2,80	0,0378	0,3692	**97,56**	0,2913	130
0,7	98,36	96,89	1,47	0,1241	0,2773	97,22	0,2407	143

Табл. 1. VGG16: результати для різних Dropout

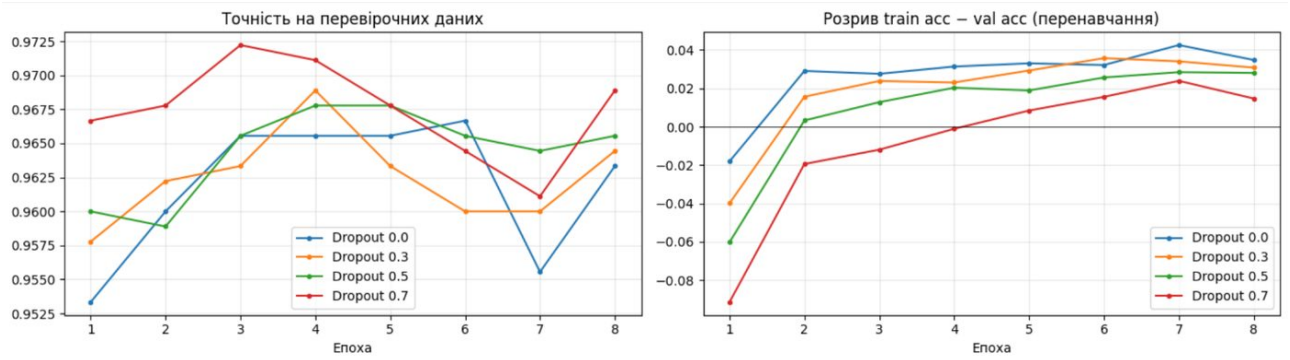


Рис. 5. VGG16: точність на перевірці і розрив train − val (міра перенавчання)

Без Dropout мережа перенавчається вже з 2-ї епохи: на навчальних даних 99,8%, на перевірочних 96,3%, а втрати на перевірці ростуть (0,39 → 0,53) — модель запам'ятовує навчальні фото. Що більший Dropout, то **менший розрив** train − val (3,5% → 3,1% → 2,8% → 1,5%) і нижчі втрати на перевірці й тесті (тест: 0,55 → 0,24). При Dropout 0,7 перші епохи навчальна точність навіть нижча за перевірочну — бо під час навчання вимикається 70% нейронів, а на перевірці працюють усі. Найкраща точність на тесті — **97,56% при Dropout 0,5**; 0,7 дає найменше перенавчання, але трохи повільніше навчається.

5. InceptionV3

Dropout	Train acc, %	Val acc, %	Розрив train−val	Train loss	Val loss	Test acc, %	Test loss	Час, с
0,0	99,95	96,89	3,06	0,0099	0,0795	96,67	0,1022	132
0,3	99,26	96,89	2,37	0,0259	0,0818	96,67	0,0970	130
0,5	98,21	96,89	1,33	0,0437	0,0819	96,22	0,0965	129
0,7	97,26	96,56	0,71	0,0723	0,0755	**96,89**	0,0912	120

Табл. 2. InceptionV3: результати для різних Dropout

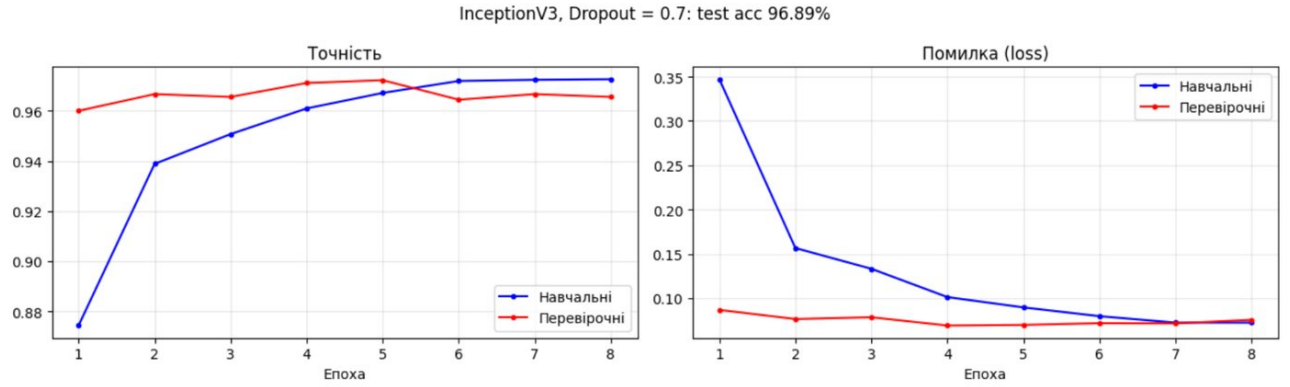


Рис. 6. InceptionV3, Dropout = 0,7 — найкраща модель (96,89%)

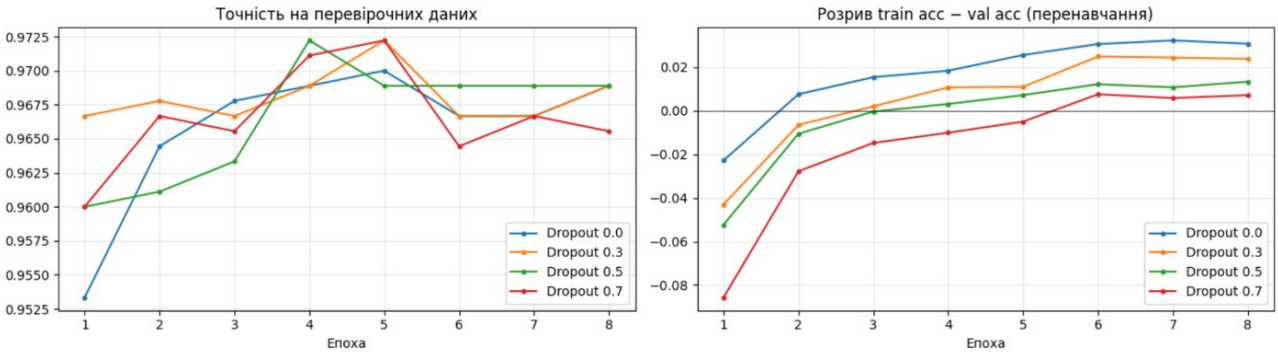


Рис. 7. InceptionV3: точність на перевірці і розрив train − val

Та сама закономірність: розрив train − val зменшується з 3,1% (без Dropout) до 0,7% (Dropout 0,7). Точність на тесті майже не залежить від Dropout (96,2–96,9%), бо після GAP класифікатор маленький (0,5 млн параметрів) і перенавчається слабше, ніж VGG16 з Flatten. Втрати InceptionV3 значно нижчі (0,08–0,10), тобто мережа впевненіша у своїх відповідях.

6. ResNet50

Dropout	Train acc, %	Val acc, %	Розрив train−val	Train loss	Val loss	Test acc, %	Test loss	Час, с
0,0	100,00	96,89	3,11	0,0072	0,0942	**97,33**	0,0699	135
0,3	99,45	97,00	2,45	0,0176	0,0821	97,11	0,0764	128
0,5	98,95	97,33	1,62	0,0312	0,0788	97,11	0,0716	124
0,7	98,10	97,11	0,98	0,0485	0,0796	96,78	0,0795	127

Табл. 3. ResNet50: результати для різних Dropout

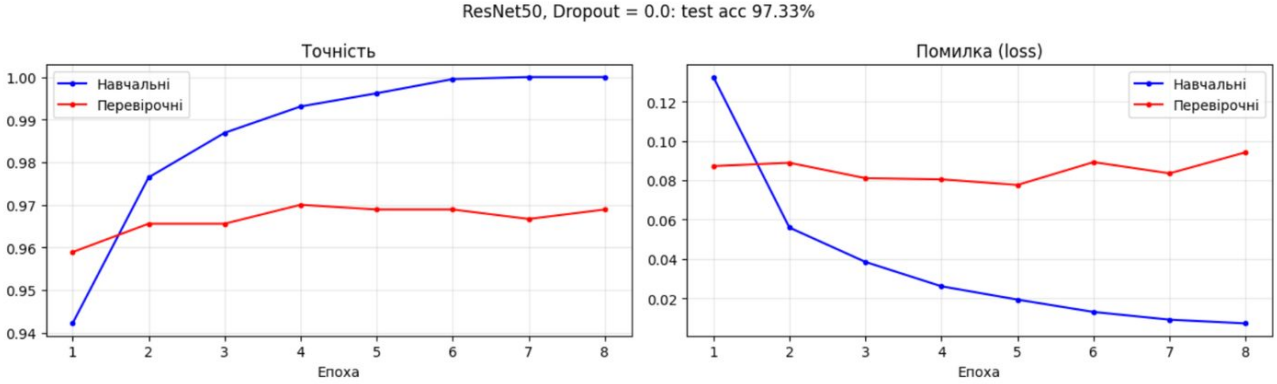


Рис. 8. ResNet50, Dropout = 0,0 — найкраща модель (97,33%)

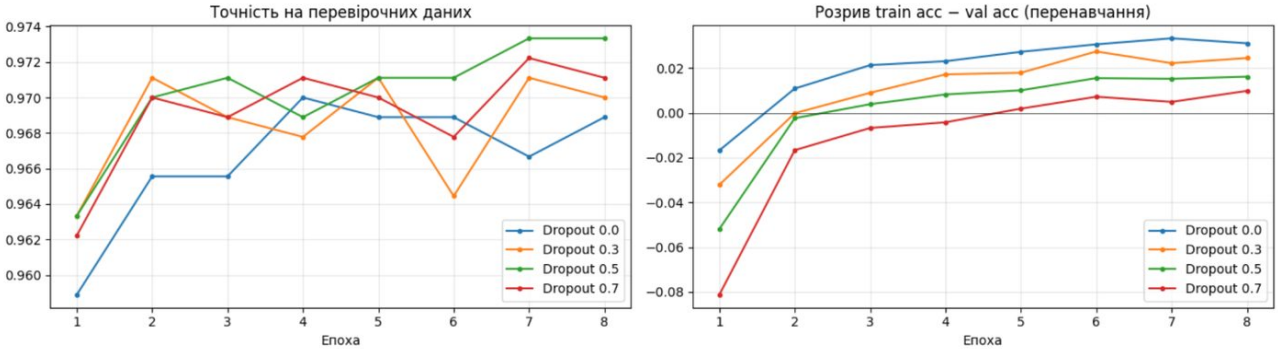


Рис. 9. ResNet50: точність на перевірці і розрив train − val

Розрив train − val знову зменшується зі збільшенням Dropout: 3,1% → 2,5% → 1,6% → 1,0%, а найкраща точність на перевірці (97,33%) — при Dropout 0,5. Найкращий результат на тесті формально дала модель **без Dropout — 97,33%**, але різниця між усіма варіантами 0,2–0,6%, тобто 2–5 зображень із 900 — це в межах випадкового розкиду. Без Dropout ResNet50 досягає 100% на навчальних даних, а втрати на перевірці з 5-ї епохи ростуть (0,078 → 0,094) — ознака початку перенавчання; при довшому навчанні Dropout дав би перевагу.

7. Розпізнавання нових зображень

Найкращі моделі перевірено на тих самих власних фото, що й у лабораторній 7: 20 звичайних (10 котів, 10 собак) і 6 «складних» — тварина у високій траві.

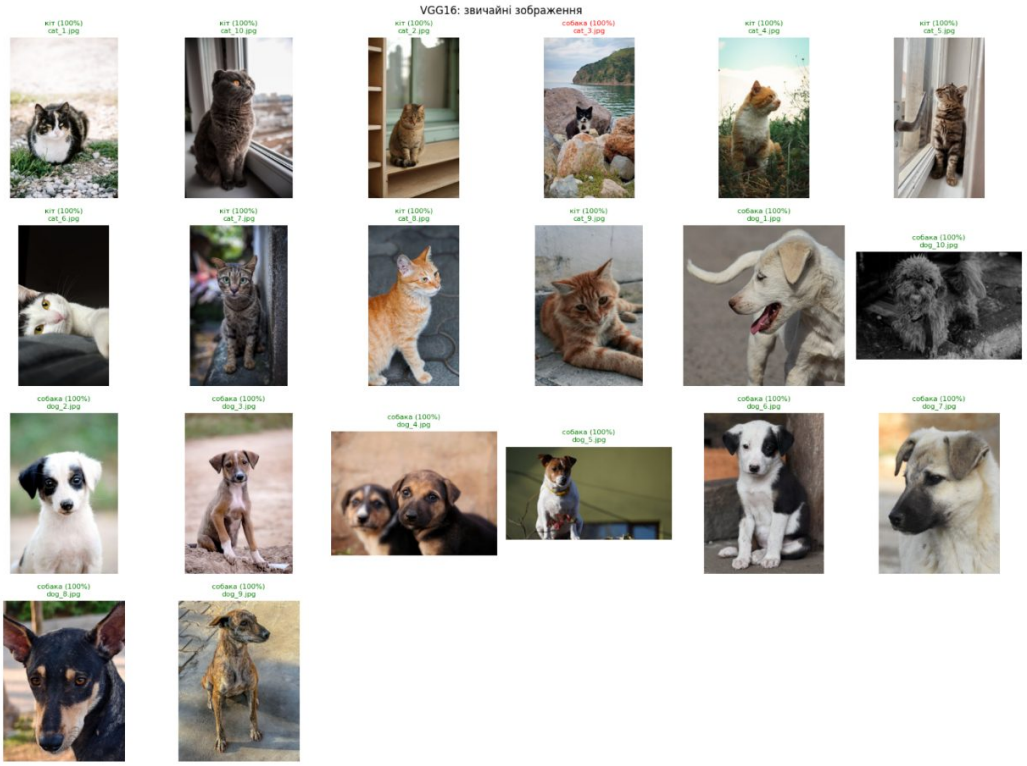


Рис. 10. VGG16: звичайні зображення — вірно 19 з 20 (помилка — cat_3)

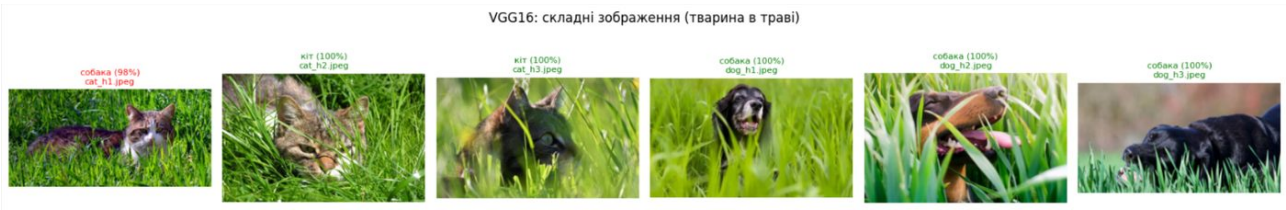


Рис. 11. VGG16: тварини в траві — вірно 5 з 6

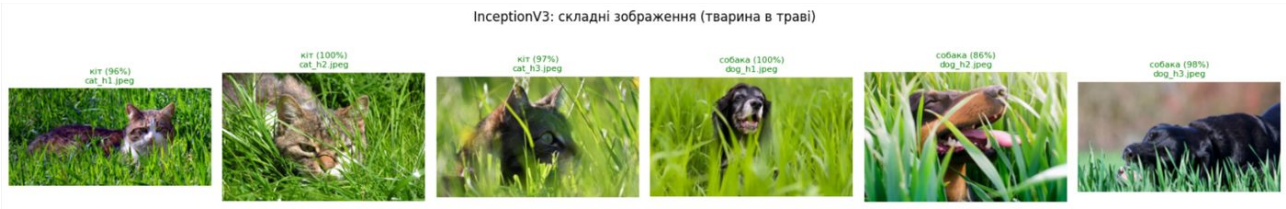


Рис. 12. InceptionV3: тварини в траві — вірно 6 з 6

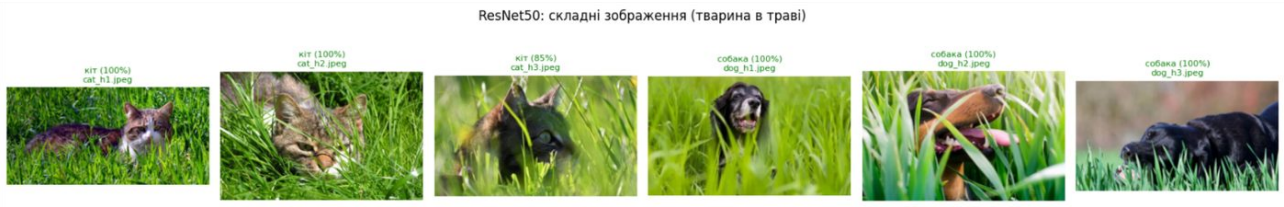


Рис. 13. ResNet50: тварини в траві — вірно 6 з 6

InceptionV3 і ResNet50 так само розпізнали **19 з 20** і помилились на тому ж фото cat_3. Різниця між мережами — у складних фото: VGG16 сплутала одного кота в траві (cat_h1 → собака 98%), InceptionV3 і ResNet50 розпізнали всі 6. Сітки звичайних фото для InceptionV3 і ResNet50 аналогічні рис. 10 (є в ноутбуках).

8. Порівняння мереж і відповідь на питання завдання

Модель	Найкращий Dropout	Test acc, %	Власні фото	Тварини в траві	Навчаються параметрів	Час на модель
CNN з лаби 7 (з нуля)	0,5	84,75	16 / 20	3 / 6	~1,2 млн	~6 хв (8 епох, 17,5 тис. фото)
VGG16	0,5	97,56	19 / 20	5 / 6	2,10 млн	~2,2 хв
InceptionV3	0,7	96,89	19 / 20	6 / 6	0,52 млн	~2,1 хв
ResNet50	0,0	97,33	19 / 20	6 / 6	0,52 млн	~2,1 хв

Табл. 4. Порівняння з лабораторною 7

Так, перенесення навчання значно покращило розпізнавання. Точність на тестовому наборі зросла з 84,75% до 96,9–97,6% (помилок у 5–6 разів менше), хоча навчальних фото було майже в 4 рази менше (4 200 проти 17 500), а навчання тривало ~2 хв. На власних фото — 19 з 20 замість 16 з 20, на складних фото в траві — 5–6 з 6 замість 3 з 6: власна CNN усіх котів у траві вважала собаками (орієнтувалась на фон), а попередньо навчені мережі «дивляться» на саму тварину, бо бачили мільйони різних фото на ImageNet. Єдине фото, на якому помиляються всі моделі, — cat_3: маленький кіт на камінні біля моря займає невелику частину кадру.

9. Збережені моделі

Найкраща модель кожної мережі збережена у форматі `.keras` і завантажена з Colab (остання комірка ноутбука), разом з CSV-таблицями результатів:

- `lab8_VGG16_best.keras` (80,2 МБ), `lab8_InceptionV3_best.keras` (90,2 МБ), `lab8_ResNet50_best.keras` (96,6 МБ);
- `lab8_<мережа>_dropout.csv`, `_own.csv`, `_summary.csv` — результати експериментів і розпізнавання.

Висновки

- Перенесення навчання дозволяє отримати високу точність на малому наборі даних і за кілька хвилин: заморожена згорткова частина з ImageNet працює як готовий виділювач ознак, а навчається лише невеликий класифікатор.
- Dropout зменшує перенавчання: що він більший, то менший розрив між точністю на навчальних і перевірочних даних (VGG16: 3,5% → 1,5%; InceptionV3: 3,1% → 0,7%) і

нижчі втрати на перевірці. Занадто великий Dropout сповільнює навчання, оптимум для VGG16 — 0,5.

- Найкраща точність на тесті — VGG16 (97,56%), InceptionV3 — 96,89%, ResNet50 — 97,33%; різниця між мережами невелика, всі суттєво кращі за власну CNN (84,75%).
- На власних і «складних» фото попередньо навчені мережі помиляються значно рідше: вони менше залежать від фону.